

LAB 1: DEVELOPMENT TOOLS OVERVIEW, ENVIRONMENT SETUP & SQL DATABASE DESIGN

Course: Web Application and Database Development Practicum (Backend & Database)

1. COURSE INTRODUCTION AND LAB REGULATIONS

a. Course Objectives

This Web Application and Database Development Practicum course equips students with the skills to build data management systems and develop professional server-side (backend) applications. Upon completing the course, students will be able to:

- Develop complete **RESTful APIs** using the **Node.js** runtime and the **Express** framework.
- Design, manipulate, and optimize relational databases (**MySQL**) and NoSQL document databases (**MongoDB**).
- Apply backend security mechanisms: password hashing (**bcrypt**), and token-based authentication and authorization (**JWT**).
- Handle real-time communication (**WebSocket** with **Socket.IO**) and schedule background jobs (**node-cron**).
- Manage application processes with **PM2**, package source code with **Git/GitHub**, and deploy backend applications to cloud platforms (**Render**, **MongoDB Atlas**, **db4free**).

b. Grading Method

Assessment Component	Weight	Notes
Attendance	20%	Full participation in all lab sessions.
In-class Lab Exercises	40%	Completed and submitted at the end of each lab session.
Homework	40%	Completed individually and submitted by the deadline.

c. Submission Regulations

- All in-class lab exercises and homework must be completed individually.

- Students must submit their work as a single Word file (**.docx**); the end of the file must include a link to the **GitHub repository** containing the source code (the repository must be set to **Public**).
- Within the Word file, students must include screenshots of the execution results (for example: terminal output, MySQL query results, Mongo Compass screens, Postman test results) for each question.
- File naming convention: **Lab<Session Number>_<Student ID>_<Full Name>.docx**

Example: Lab1_26521234_NguyenVanA.docx

2. DEVELOPMENT ENVIRONMENT SETUP GUIDE

a. Visual Studio Code (VS Code)

The official source code editor for this course. Download link:

<https://code.visualstudio.com/download>

Recommended extensions to install in VS Code:

- **ESLint & Prettier:** Automatically format code and check for JavaScript syntax errors.
- **Thunder Client / Postman:** API testing tool directly inside VS Code.
- **GitLens:** Helps manage and visualize Git commit history.
- **MySQL / MongoDB for VSCode:** Manage and query databases directly within VS Code.

b. Node.js & npm

Node.js is the server-side JavaScript runtime environment. Download the **LTS (Long Term Support)** version. Download link: <https://nodejs.org/en/download>

c. Git (Version Control)

Source code version control tool. Download link: <https://git-scm.com/downloads>

d. Local Server and Database Toolkit (MySQL & MongoDB)

- **MySQL Server & MySQL Workbench:** Administration tool and client for interacting with the MySQL database. Download at <https://dev.mysql.com/downloads/>
- **MongoDB Community Server & Mongo Compass:** NoSQL document database system and its management interface, MongoDB Compass. Download at <https://www.mongodb.com/try/download/community>

e. Cloud Service Account Setup (one-time for the entire course)

- **MongoDB Atlas (Cloud NoSQL):** Create a free account at <https://www.mongodb.com/cloud/atlas>, create a free M0 cluster, and obtain the connection string (URI) in the format **mongodb+srv://...**
- **Render.com (Cloud Backend API Hosting):** Sign up for a free account using your GitHub account at <https://render.com> to deploy your Node.js API.

f. API Testing Tool: Postman

Download the Postman Desktop Agent or install the Postman app. Download link:

<https://www.postman.com/downloads/>

g. Verify the Environment Setup

Open Terminal (on macOS/Linux) or Command Prompt/PowerShell (on Windows) and run the following commands one by one to check the installed versions:

```
code --version
git --version
node --version
npm --version
```

Expected result: All of the commands above should print a specific version number (e.g., v20.x.x, git version 2.4x.x). If you see a “Command not found” error, check your PATH environment variable or restart your computer.

3. SOURCE CODE MANAGEMENT WITH GIT & GITHUB

a. Configure Your Personal Git Information

Run the following commands in Terminal to set up your commit name and email:

```
git config --global user.name "Nguyen Van A"
git config --global user.email "nguyenvana@gmail.com"

# Check the configuration:
git config --list
```

b. Workflow for Pushing Code to a GitHub Repository

- **Step 1:** Go to <https://github.com> and create a new repository named **web-backend-lab1**, set to Public.
- **Step 2:** Open your project folder on your computer in VS Code, open the Terminal, and run the following command sequence:

```
echo "# Web Backend & Database Lab 1" >> README.md
git init
git add .
git commit -m "First commit - Lab 1 setup"
git branch -M main
git remote add origin https://github.com/<your-username>/web-backend-lab1.git
git push -u origin main
```

4. IN-CLASS EXERCISES

Exercise 1: Build Your First Node.js Backend Application and Configure package.json

Requirement: Create the project folder **lab1-backend**, initialize a Node.js project, install the **express** and **dotenv** libraries, and write an HTTP server listening on port **5000**.

Terminal commands:

```
mkdir lab1-backend
cd lab1-backend
npm init -y
npm install express dotenv
```

Sample server.js code:

```
// server.js - Simple Express Server
const express = require('express');
const app = express();
const PORT = process.env.PORT || 5000;
```

```
// Middleware to read JSON data
app.use(express.json());

// Health-check endpoint
app.get('/', (req, res) => {
  res.status(200).json({
    message: 'Welcome to Web Backend & Database Lab 1 API!',
    status: 'Success',
    timestamp: new Date()
  });
});

app.get('/api/health', (req, res) => {
  res.status(200).json({ status: 'OK', uptime: process.uptime() });
});

const server = app.listen(PORT, () => {
  console.log(`Server is running at: http://localhost:${PORT}`);
});

server.on('error', (err) => {
  if (err.code === 'EADDRINUSE') {
    console.error(`[ERROR] Port ${PORT} is already in use. Please choose a different port or close the application currently using it.`);
  } else {
    console.error(`[ERROR] Failed to start server:`, err.message);
  }
  process.exit(1);
});
```

Run **node server.js** and open **http://localhost:5000** in your browser to check the result.

Extended requirement: Add a new API endpoint using the GET method at the path **/api/greeting** that returns the student's personal information in JSON format.

```
{
  "success": true,
  "message": "Hello, this is my information",
  "student": {
    "fullName": "Nguyen Phan Thao Nguyen",
    "studentId": "26110906",
    "class": "26DTHC2"
  },
  "timestamp": "2026-09-11T08:51:41.247Z"
}
```

Example response from /api/greeting

Exercise 2: Design a Schema & Execute SQL DDL/DML on a MySQL Server

Requirement: Connect to MySQL using MySQL Workbench. Create the **ecommerce_db** database, build two tables: **users** and **products**, with primary keys and constraints, insert sample data, and run some queries.

Sample DDL & DML script:

```
-- 1. Create the database
CREATE DATABASE IF NOT EXISTS ecommerce_db;
USE ecommerce_db;

-- 2. Create the users table (DDL)
CREATE TABLE IF NOT EXISTS users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  full_name VARCHAR(100) NOT NULL,
  email VARCHAR(100) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  role ENUM('admin', 'user') DEFAULT 'user',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 3. Create the products table (DDL)
CREATE TABLE IF NOT EXISTS products (
  id INT AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(150) NOT NULL,
  price DECIMAL(10, 2) NOT NULL,
  stock INT DEFAULT 0,
  category VARCHAR(50),
```

```

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 4. Insert sample data (DML - INSERT)
INSERT INTO users (full_name, email, password_hash, role) VALUES
('Nguyen Van Admin', 'admin@gmail.com', 'hashed_pwd_123', 'admin'),
('Tran Thi User', 'user@gmail.com', 'hashed_pwd_456', 'user');

INSERT INTO products (title, price, stock, category) VALUES
('Laptop Dell XPS 15', 35000000.00, 10, 'Electronics'),
('Keychron K2', 2200000.00, 25, 'Accessories'),
('Logitech MX Master 3S', 2500000.00, 15, 'Accessories');

```

Extended requirement: Extend the database structure to manage an **Orders** feature and run realistic queries.

1. Extend the Database Structure (DDL)

Write an SQL script that creates 2 new tables linked by foreign keys:

1. **orders** table (order management):

- **id**: INT, Primary Key, Auto Increment.
- **user_id**: INT, Foreign Key referencing **users(id)** (*NOT NULL*).
- **total_amount**: DECIMAL(10, 2), total order value (*NOT NULL*).
- **status**: ENUM('pending', 'completed', 'cancelled'), defaults to 'pending'.
- **created_at**: DATETIME, defaults to the current time (**CURRENT_TIMESTAMP**).

Example:

id	user_id	total_amount	status	created_at
1	1	27400000.00	completed	2026-09-11 16:19:28
2	2	1800000.00	pending	2026-09-11 16:19:28
3	1	8500000.00	completed	2026-09-11 16:19:28
NULL	NULL	NULL	NULL	NULL

2. **order_items** table (order line items):

- **id**: INT, Primary Key, Auto Increment.
- **order_id**: INT, Foreign Key referencing **orders(id)**.

- **product_id**: INT, Foreign Key referencing **products(id)**.
- **quantity**: INT, quantity purchased (*must be > 0*).
- **price**: DECIMAL(10, 2), unit price at the time of purchase.

Example:

id	order_id	product_...	quantity	price
5	1	1	1	25000000.00
6	1	2	1	2400000.00
7	2	3	1	1800000.00
NULL	NULL	NULL	NULL	NULL

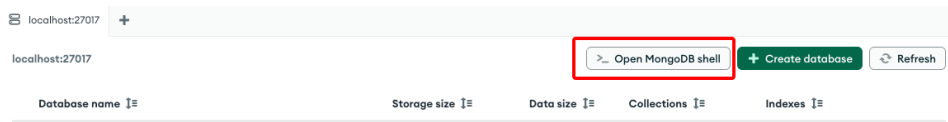
2. Data Manipulation & SQL Queries (DML & DQL)

Write SQL statements to solve the following four requirements:

- **Q1 (Insert sample data):**
 - Insert at least 3 orders into the orders table for different users.
 - Insert the corresponding product details into the order_items table.
- **Q2 (Filter & search products):**
 - Write a **SELECT** statement to retrieve the list of products priced between 100,000 VND and 1,000,000 VND, sorted by price from highest to lowest (**ORDER BY ... DESC**).
- **Q3 (Table joins - JOIN):**
 - Write an **INNER JOIN** statement linking the **users**, **orders**, **order_items** and **products** tables to display a detailed order report with the columns:
Order ID | Customer Name | Product Name | Quantity | Unit Price | Order Status
- **Q4 (Revenue statistics - GROUP BY & AGGREGATE):**
 - Write a statement to calculate the total revenue from all completed orders (**status = 'completed'**).
 - Write a statement to count the number of orders placed by each user (**GROUP BY user_id**).

Exercise 3: Working with MongoDB NoSQL Documents Using Mongo Compass / Shell

Requirement: Connect to a local MongoDB instance (**mongodb://localhost:27017**) or MongoDB Atlas Cloud. Create the **shop_db** database and the **orders** collection. Insert a document containing nested arrays (embedded sub-documents) and run a search query.



Sample MongoDB Shell / Compass query commands:

```
// 1. Switch to the shop_db database
use shop_db;

// 2. Insert 2 order documents into the orders collection (insertOne / insertMany)
db.orders.insertMany([
  {
    order_code: "ORD-2026-001",
    customer_name: "Nguyen Van A",
    customer_email: "nguyenvana@gmail.com",
    total_amount: 37500000,
    status: "completed",
    items: [
      { product_name: "Laptop Dell XPS 15", quantity: 1, price: 35000000 },
      { product_name: "Logitech MX Master 3S Mouse", quantity: 1, price: 2500000 }
    ],
    created_at: new Date()
  },
  {
    order_code: "ORD-2026-002",
    customer_name: "Tran Thi B",
    customer_email: "tranthib@gmail.com",
    total_amount: 2200000,
    status: "pending",
    items: [
      { product_name: "Keychron K2 Mechanical Keyboard", quantity: 1, price: 2200000 }
    ],
    created_at: new Date()
  }
]);

// 3. Query documents (Find)
// Find orders with status 'completed'
db.orders.find({ status: "completed" }).pretty();
```

```
// 4. Update an order's status (Update)
db.orders.updateOne(
  { order_code: "ORD-2026-002" },
  { $set: { status: "processing" } }
);
```

Extended requirement: Array queries and aggregation in MongoDB

After creating the `shop_db` database and inserting the first 2 orders, perform the following extended tasks using the MongoDB Shell (mongosh) or MongoDB Compass:

1. Q1 (Insert varied data)

- Insert 2 new orders into the **orders** collection with the codes **ORD-2026-003** (status 'completed', with 3 products in the **items** array) and **ORD-2026-004** (status 'cancelled').

2. Q2 (Conditional & nested-document queries)

- Query for orders with a **total_amount** of 5,000,000 VND or more **AND** a status of 'completed'.
- Query for all orders that include the product “Logitech MX Master 3S Mouse” in the **items** array.

3. Q3 (Array update)

- Add a new product { **product_name**: "XL Gaming Mouse Pad", **quantity**: 1, **price**: 200000 } to the **items** array of order **ORD-2026-002** (using the **\$push** operator).
- At the same time, add 200,000 VND to the **total_amount** field of this order (using the **\$inc** operator).

4. Q4 (Aggregation Framework statistics)

- Write an aggregation command (**\$match**, **\$group**) to calculate the total revenue from all completed orders ('completed').
- Write an aggregation command to count the number of orders grouped by status ('**status**').

5. HOMEWORK EXERCISES

Complete the homework, capture screenshots of the execution results, and push all source code to the GitHub repository created in Section 3.

Homework 1: Design a Library Management Database

Write a DDL SQL script to create the **library_db** database with 3 tables linked by foreign keys:

- **authors** table (id, author_name, nationality)
- **books** table (id, title, author_id [FK], category, publish_year)
- **borrow_records** table (id, book_id [FK], borrower_name, borrow_date, return_date)
- Insert at least 3 sample records into each table.
- Write an SQL **JOIN** statement across all 3 tables to display: Book Title, Author Name, Borrower Name, and Borrow Date.

Homework 2: Build and Manage a Blog System NoSQL Database on MongoDB

Use MongoDB Atlas (Cloud) or a local Mongo Compass / MongoDB Shell instance to complete the following:

- Create the **blog_db** database and the **posts** collection.
- Insert at least 3 posts (documents) containing embedded objects and arrays, with the following structure:
 - **title** (String), **category** (String), **views** (Number).
 - **author**: object containing **name** and **email**.
 - **tags**: array of keywords (e.g., ["nodejs", "express", "backend"]).
 - **comments**: array of comment objects (**user**, **content**, **created_at**).

Write a MongoDB Shell script to run the following queries:

- **Query 1:** Find posts in the 'Tech' category **AND** with views of 100 or more.
- **Query 2:** Find all posts tagged with 'nodejs'.
- **Update:** Add a new comment to the comments array of a specific post by title (using the **\$push** operator) and increment its views count by 1 (using the **\$inc** operator).
- **Aggregation:** Calculate the total number of views (**totalViews**) and the total number of posts (**totalPosts**), grouped by category.

Homework 3: Deploy a Node.js “Hello World” Application to a Cloud Service (Render.com)

- Push the entire **lab1-backend** Node.js project from Exercise 1 to your personal GitHub repository.

- Log in to <https://render.com>, create a new **Web Service**, and connect it to the GitHub repository you just created.
- Configure the Build Command as **npm install** and the Start Command as **node server.js**.
- After Render successfully deploys the app, copy the public URL (e.g., **https://lab1-backend-xyz.onrender.com**) into your submitted Word file and test the API endpoint.